Mini-Katran: Load-Aware, Stateless L4 Load Balancing with eBPF at TC

CS 331 Computer Networks, 2026-27 Semester 1 · Project 16 · Team T020
Akshit Chhabra (24110026), Bhavay Goyal (24110070), Devarshi Savalgi (24110103), Kunal Chandola (24110176)
6 November 2026 · github.com/akshit-codes/CS331-T020-MiniKatran

Abstract. An L4 load balancer must keep each connection on one server, so most give servers a fixed share. That wastes capacity when servers differ and misses latency targets when one slows down. We built a load balancer in the style of Cloudflare's Unimog: a stateless director at the TC hook hashes packets into 4096 buckets, a feedback controller moves buckets according to measured load, and a redirector on each server forwards packets of older connections to their previous server. On one laptop with four unequal backends, tables found by the controller reach about 91–93% of the summed capacity at p99 ≤ 20 ms, against 95–97% for shares proportional to measured capacity and 62.5% for equal shares. Moving half of the buckets broke 0 of 300 long-lived connections with second hops and 130–146 without. A static table built for a known disturbance beats every controller; the controller's value is that it recovers without knowing the disturbance. A single quick run on four real laptops over a phone hotspot confirmed connection survival, but no configuration recovered within the latency target under a noisy neighbour. Most results rest on one to three runs, and we say so where they appear.

1 Introduction

A layer-4 load balancer sees packets, not requests. It can choose a server only when a connection starts and must then keep sending that connection's packets to the same place. Designs such as Maglev [1] and Katran [2] spread connections evenly, or by fixed weights, using consistent hashing. If servers have unequal capacity, or one of them slows down because of a noisy neighbour or an expensive request type, an even split overloads the weakest server while others sit partly idle.

Changing the split while traffic is flowing is the hard part. If the mapping from flows to servers changes, packets of existing connections can reach a server with no socket for them, which answers with a reset. A per-connection table on the balancer avoids this at the cost of state that must be kept and is lost on restart. We wanted a balancer with no per-connection state that can still move load safely. Our workload is CPU-bound HTTP over TCP on servers of unequal speed, and our metric is how much of the servers' total capacity can be used while staying within a latency target. Four research questions guided the work:

- **RQ1:** What fraction of the servers' summed capacity can each policy use within the SLO (p99 ≤ 20 ms, errors ≤ 0.1%)?
- **RQ2:** For which disturbance timescales does a dynamic controller do better than a static table?
- **RQ3:** Which load signal should it use: CPU utilisation, requests in flight (RIF), cgroup CPU or pressure stall information (PSI)?
- **RQ4:** Do connections survive table changes?

2 Background and related work

Unimog [3] is the design we follow: flows hash into a table of buckets, each bucket names a first and second server, servers forward packets they don't recognise to the second hop, and a control loop adjusts bucket counts using processor utilisation. It uses a forwarder on every server; we use one dedicated balancer so forwarding doesn't mix into the CPU we measure. Second-hop forwarding comes from **Beamer** [4], which also has generation numbers and longer chains that we leave out. The idea of swapping a bucket's two hops is Unimog's,

and almost all of our moves are swaps. Rendezvous hashing for the initial hops comes from **GLB** [5]. **Maglev** [1] gives the general idea of a large lookup table, though not its filling method or connection tracking.

Prequal [6] argues against balancing on CPU and picks a replica per request from fresh probes of RIF and latency. We test RIF as a signal, but moving buckets of new connections every one or two seconds is a different mechanism, so our comparison is not a test of Prequal. The XDP paper [7] and Katran motivated hooking early in the packet path. Our measurement rules draw on Heiser's benchmarking crimes [8] and on "The Tail at Scale" [9].

What is ours in this project: running the balancer at TC on laptops with no native XDP, one noise-based tuning rule applied to every signal, and the comparison of CPU and RIF on unequal cores within one bucket controller.

3 Design

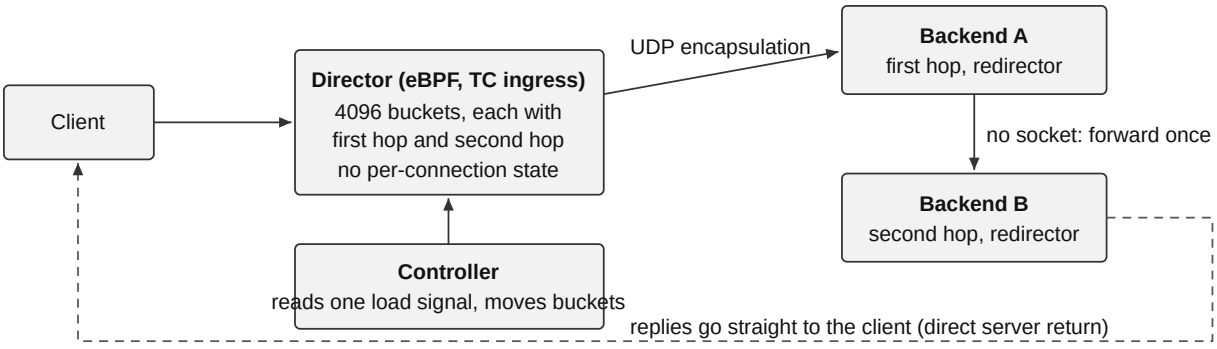


Figure 1: Architecture.

Director. It hashes each TCP packet's 4-tuple (jhash, fixed seed) to a bucket. The bucket gives a first and a second hop. The director prepends an outer IPv4 header, a UDP header (port 7700, source port derived from the flow hash, checksum 0) and a 12-byte header carrying version, flags, bucket, second hop and table generation, then sends the packet to the first hop. The director keeps no state per connection. A new table is built in full and swapped in with one map update, so a packet sees the old table or the new one, never a mix.

Redirector. On each backend's TC ingress it decides whether to keep or forward a packet. It keeps the packet if it has already been forwarded (the HOPPED flag), if the bucket has no second hop, if it is a SYN, if a socket other than a listener exists, or if a listener matches and the packet carries a valid SYN cookie. Otherwise it sets HOPPED and forwards once to the second hop. Because SYNs and HOPPED packets are always kept, a packet cannot loop. The SYN-cookie check matters: with it switched off, a move broke 632–671 of about 5950 new connections.

Why a swap is safe. Exchanging a bucket's hops turns [A|B] into [B|A]. New connections go to B. Packets of older connections land on B, which has no socket and forwards them to A. The pair of servers doesn't change, so repeated swaps can't strand a connection, but only because the redirector forwards. A real capture shows the hop (.10 is the LB, .11 and .12 are backends); only the outer addresses and the flag change:

```
outer .10 > .11  flags 0x00           bucket 879  second_hop .12 | inner 10.0.0.2:40568 >
VIP:8080
outer .11 > .12  flags 0x01 (HOPPED)  bucket 879  second_hop .12 | inner 10.0.0.2:40568 >
VIP:8080
```

Controller. Each period it reads one signal per backend and sets a target bucket count `n·(mean/u)^0.3`, limited to a 25% step and a floor of 16 buckets. It acts only outside a dead band and above a low-load gate, moves at most 2% of the table per period, and prefers swaps. A bucket whose hops were reassigned stays frozen for 20 s, so that connections shorter than that can't lose both hops.

Signals and tuning. The signals are whole-core CPU, RIF per core, total RIF per server, cgroup CPU and PSI. To avoid comparing our tuning effort instead of the signals, every signal uses one rule: measure its noise at averaging periods of 1, 2, 4 and 8 s, set the dead band to a multiple of that noise, and choose the period in a calibrated simulator for fast recovery at low churn.

Scope. TCP over IPv4, one VIP, two hops, and no health checks. There is no UDP because the keep-or-forward decision depends on TCP state, a socket or a SYN cookie. UDP has no handshake, and an unconnected socket can't tell one flow from another. Supporting it would take connected UDP sockets or QUIC connection IDs, which is a different keep rule. **Why TC and not XDP:** our laptops are Wi-Fi only with no NIC offering native XDP, so XDP would always run in generic mode on real hardware.

4 Implementation

The data path is about 750 lines of restricted C. The director uses `bpf_skb_adjust_room` with the L3, L4 and L2-size encapsulation flags, `bpf_skb_change_tail` to trim padding, and `bpf_redirect`. The table is an `ARRAY_OF_MAPS`. The redirector uses `bpf_skc_lookup_tcp`, `bpf_tcp_check_syncookie` and `bpf_redirect_neigh`. A tracepoint program counts resets, which gives an independent count of broken connections. The control plane, backend and load generator are about 4200 lines of Go using cilium/ebpf, with about 1000 lines of tests, and the harness and analysis are about 3000 lines of Python. An XDP twin of the director shares its logic and exists only for a cost comparison.

Tests include unit tests, golden-packet tests that run the BPF programs in the kernel (32 passing cases and subtests, comparing output bytes on both hooks), and a 20-check end-to-end test. The simulator checks the controller's safety rules at scale: 0 broken connections in about 173 M simulated connections per noise level with protection, and 27–6711 without, so the checker does fire.

5 Methodology

One-laptop testbed. Network namespaces on an i7-14650HX with turbo off and fixed clocks, the load generator, router and backends pinned to separate CPUs, and each server's request work on dedicated cores. The four backends are beP1 (2 P-cores, slowed 1.6×) at 957 req/s, beP2 (2 P-cores) at 1656, and beE1 and beE2 (4 E-cores each) at 1810 each, summing to 6234 req/s. A request costs 1 ms of P-core CPU on average, exponentially distributed; an E-core takes 1.85× as long. Load is open-loop, 512 connections of 20 requests each, plus 100 long-lived connections in the dynamic runs.

Metrics. Usable capacity fraction is the highest rate meeting the SLO through the balancer, found by bisection on 20-s probes, divided by the sum of the backends' capacities measured alone. Recovery is the time until p99 stays within the SLO for three 5-s windows. Broken connections are counted three ways, by load generator, reset monitor and backend counters, and these agree.

Names. We call the table proportional to capacities measured before any disturbance the *oracle*, though nobody searched for the best split and some controller tables measured above it. *Hindsight* is a table proportional to the capacities during one known disturbance.

Where we fell short. Our own protocol asked for at least 5 capacity searches, at least 10 fixed-rate runs, and no single-run results. We ran 1–3 runs per configuration. The smallest difference we claim is about 5%, twice a 2.5% run-to-run spread from only 4 pairs of probes; its 95% interval runs from 3% to 15%, so it is a rule of thumb. Near the SLO, p99 isn't monotonic with load. Most sessions weren't fully interleaved; only the last two were. Several tests were added after we saw data (total RIF, the contending neighbour, the timescale sweep, the forwarding-off contrast), so those are exploratory.

6 Results on one laptop

RQ1: usable capacity

Policy	Runs and fraction, first session	Runs and fraction, interleaved session
Equal shares	1: 62.5%	–
Shares by core count	1: 86.7%	–
Oracle (proportional to capacity)	3: 95.1% (94.0–96.2)	2: 96.7% (96.2–97.1)
Table found by CPU controller	2: ≥ 92.8%	3: 92.7% (87.2–100.1)
Table found by total-RIF controller	3: ≥ 92.8%	3: 90.9% (88.7–93.4)
Table found by RIF-per-core controller	2: 86.6% (84.9–88.3)	–
Oracle's shares, no balancer	–	2: 100.0% (99.1–100.9)

Table 1: Capacity at p99 ≤ 20 ms as a fraction of 6234 req/s. "≥": a run hit the top of its search range.

By-cores loses 8.4 points to the oracle because an E-core isn't worth a P-core. Controller tables come close to the oracle, possibly a few points below. Sending the oracle's shares directly reaches about 100%, and with the other backends busy capacity falls only about 2%, so at most about half of the oracle's gap is contention. Most of the rest, about 3.3%, appears only through the balancer. Requests also arrive less evenly through it, which fits hashing unevenness, but we didn't separate the causes.

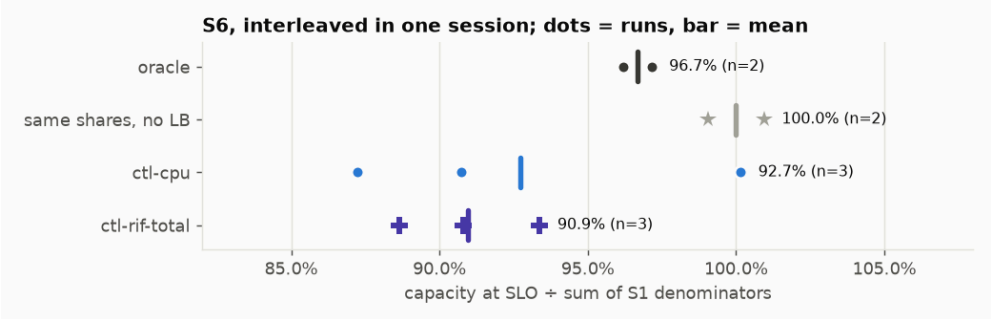


Figure 2: The interleaved session. Each dot is a run.

RQ4: connection survival

- Moving 50% of the buckets broke **0 of 300** long-lived connections with second hops, and **130–146 of 300** without (one run per hook). The live demo script gave 0 in 7 of 7 runs with second hops and 133–156 without.
- Controller runs on the final code: **0 of 400** broken in 4 antagonist runs, with 6233 moves, all swaps, and 4.5 M short requests without an error. In 34 earlier runs, before a startup fix for a race they didn't hit, 0 broke across about 75,000 moves.
- With redirectors not forwarding (1 run), 36 long-lived connections broke, with 196 failed requests overall.
- Of the 74,928 moves in the earlier runs, 74,899 were swaps. With a second hop in every bucket, the controller almost never needs the risky kind of move. We didn't test reassignment-heavy changes live.

RQ2: dynamic against static

Disturbance (75% load)	Table for before it (1 run)	Table built for it (2)	CPU controller (3)	Total RIF (2)
Step: beP2 needs 2× CPU per request	collapses: 12% errors, p99 ~17 s	0 s, 0–4% of windows over SLO	30–35 s, 33–50%	40–45 s, 50%
Neighbour on beE1's cores for 120 s	never, 100%, p99 ~206 ms	0 s, 0–4%	10 s, 17–21%	10 s, 8–12%

Table 2: Recovery time and share of 5-s windows over the SLO.

A static table that knows the disturbance beats every controller, with no transient and no moves. So the controller doesn't find something a static table can't reach. What it gives is not having to know which server, when, or how much. Hindsight tables only suit one disturbance, and from their shares they should give up capacity when nothing is wrong (85–90%, computed, not measured). In a flapping antagonist with periods of 5, 20 and 60 s, the out-of-date table spent 50–96% of windows over the SLO and the CPU and total-RIF controllers 8–15%, with RIF per core at 38% at 20 s (mostly one run per point). Our hypothesis that fast disturbances defeat a controller stays half-tested: the fastest on-phase was 2.5 s, just above the 1–2 s control period and the 2.2 s connection lifetime.

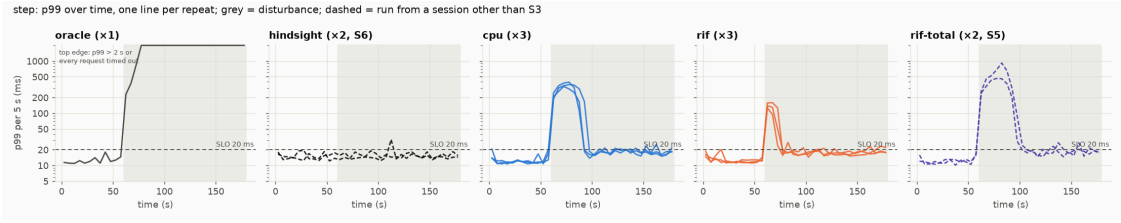


Figure 3: p99 over time around the step. Grey marks the disturbance; dashed lines are runs from another session.

RQ3: which signal

- **CPU and total RIF** perform alike: 92.7% against 90.9% (3 runs each), with no difference we can resolve. After the step, CPU settles (0–143 moves/min), while total RIF keeps moving buckets (756–1046 moves/min), though nothing breaks.
- **RIF per core** recovers fastest from the step (15–25 s) but loses about 6 points of capacity and hunts most. At equal utilisation a 4-core backend has a shorter queue per core, so it looks lighter than it is, while its slower cores miss the target first. Dividing by cores was our own choice, and this explanation came after the data.
- **cgroup CPU** doesn't see a neighbour in another cgroup (0.91×), and a controller using it never moved load away (1 run).
- **PSI** depends on where the server queues. With our worker pool it read 0.05 even at 130% load. With a backend that queues in the kernel it reached 0.99.
- A neighbour on sibling hyperthreads slowed work only if it thrashed caches. RIF rose 25% and CPU about 10%, which is inside the controller's dead band, so no controller would act on it (1 run).

The TC and XDP hooks showed no sign of TC costing more on veth (39 vs 45 and 64 vs 78 ticks of 10 ms, 1 run, including router work). This backs running in TC on our hardware and says nothing about NICs with real native XDP.

7 Results on four real laptops

We ran a shortened campaign on physical machines over a phone hotspot, driven over SSH: an i7 as the client, a thin-and-light i5 as the LB, and two i5 laptops with two backends each. All four kernels were 7.x, on a mix of Ubuntu and Arch. This was one run per configuration with a 90-s disturbance. In the first dry run, with 20-ms

requests, capacity through the balancer came out well under the sum of the backends, and we could not rule out the hotspot as the limit. Idle ping between laptops was about 140 ms on average. We therefore doubled the request cost to 40 ms and the SLO to 800 ms, and used 32 connections. These numbers shouldn't be put in one table with the one-laptop results. Backend capacities were 28, 48, 34 and 34 req/s, 146 in total. The laptops have similar hardware, but with clocks fixed h2 ran at about 0.72 of h1's speed, which explains the lower capacity of its two backends. We don't know why the fixed clocks differ.

Under the neighbour	p99 before	Late p99	Windows over SLO	Requests failed once sent	Long-lived broken (of 20)	Moves
Oracle table	415 ms	12.9 s	89%	0	0	0
CPU controller	538 ms	2.9 s	100%	1	0	1223
Total-RIF controller	443 ms	10.0 s	100%	1	0	511
CPU controller, no forwarding	460 ms	5.4 s	94%	19	**12**	1409

Table 3: Real laptops, 1 run each. All moves were swaps. Separately, 281, 248 and 200 requests of the oracle, total-RIF and no-forwarding runs were still queued unsent when the run ended.

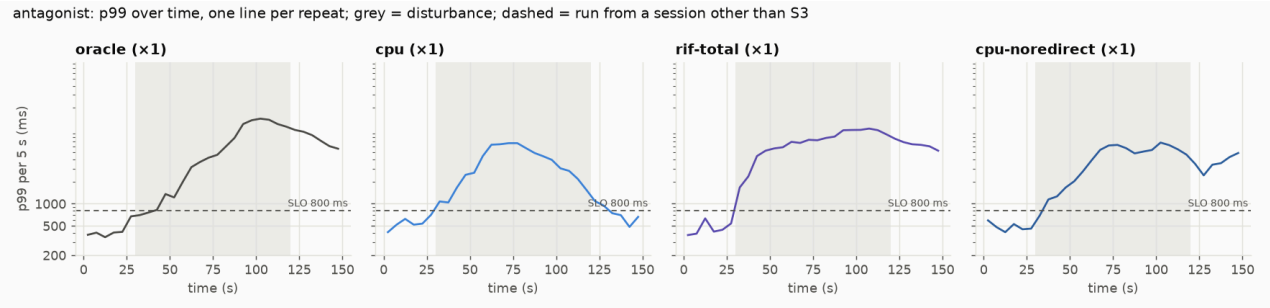


Figure 4: Real laptops. p99 per 5 s under the neighbour (grey), log scale; dashed line is the 800 ms SLO.

The mechanism works on real hardware: the director at TC on real Wi-Fi drivers, second hops crossing the air, and replies from the VIP passing the access point. Connection survival held: 0 of 20 long-lived connections broke with forwarding (1223 and 511 moves), against 12 of 20 without (1409 moves, 34 backend resets). The CPU controller also found a table by itself, 733 moves from equal shares to within 1.7 points of the oracle's.

No configuration recovered within 800 ms while the neighbour ran, against 10 s on one laptop. The CPU controller moved beE1 from 968 to 613 buckets and its p99 was falling, down to 1.5 s when the neighbour ended and under 800 ms about 10–15 s later. The late p99 figures show the direction only: most of that time is waiting for one of the 32 client connections, so the sizes aren't solid. RIF's measured noise was high (0.6–0.7), though CPU's noise also rose about fourfold and RIF averages only 1–3 per backend, so this isn't shown to be a Wi-Fi effect. The tuning rule gave RIF an 8-s period and a wide dead band, and it made no moves at all while finding its table.

Capacity fractions are usable only as "in this one run": oracle 91.8%, CPU 87.8% (search spacing 2.4%), and equal and total RIF between 80% and 102% (spacing 27.7%). Since RIF never moved, those last two are the same table measured twice, and the noise between them is larger than the gap between oracle and CPU. So we don't claim the oracle beats equal or CPU. At 40 ms the limit looks like the backends, since a queueing model with no network predicts p99 of about 860 ms at 134 req/s where we measured 695 ms; Wi-Fi shows up as noise. We read this run as evidence that the design survives a real network, not as a measurement of controller performance.

8 Non-functional testing

	Measured	Not measured
Performance	Capacity at the SLO, open-loop p99, recovery time	Maximum LB throughput, controller overhead
Reliability	Connection survival through table changes; dead controllers are detected and the run marked invalid	Backend or LB failure. No health checks. From reading the code, not a test, the CPU controller would give a dead backend more buckets because its cores look idle
Scalability	Nothing beyond 4 backends and 1 LB	More backends, several LBs
Portability	Three kernels, clang 21 and 23, Ubuntu and Arch, real Wi-Fi drivers	AMD laptops, wired networks
Repeatability	Calibration within 0.4% across nine sessions; capacity varies about 2.5% between runs (4 pairs)	The 5+ repeats our protocol asked for

9 Problems we ran into

- **A test that couldn't fail.** Our planned "no protection" run removed dwell and caps and broke nothing, because all 16,311 of its moves were swaps. That exposed the wrong test and led to our main insight: swaps are safe only because the redirector forwards. The real contrast turns forwarding off.
- **The tool was the problem.** The static comparison took four attempts. Oracle capacity first ranged from 3739 to 5452 req/s. Abandoned requests leaked between probes, the load generator ran out of CPU, and client ports stuck in TIME_WAIT stalled new connections. Our first explanation of the last one was wrong; a direct test (2 of 5 probes failing before the fix, 5 of 5 passing after) found it. Afterwards repeats agreed within 1.2%.
- **XDP to TC.** We started in XDP and hit checksum offload failures, then recognised that none of our laptops has native XDP. Moving to TC also made large offloaded packets work.
- **Claims that failed review.** "No sustained oscillation" was false. A sweep point was mislabelled. A safety test still passed with the dwell rule removed, because the controller's oldest-first selection never reached a recent bucket. "The controller beats the oracle" was only true of an out-of-date table.
- **Real machines.** Ubuntu's clang 21 emitted a forward-declared type in a nested map definition, which the loader refused, and the driver left its results owned by root. Emulation had hidden the second, and our single compiler the first.

10 Limitations, future work and conclusion

All detailed results come from one laptop, one synthetic workload and one load level for the dynamic runs, with few repeats. No true byte-heavy elephant flows were tested, since our long-lived connections carry only about 4% of requests. We never tested a disturbance too fast for the controller, or a backend failure. Live churn was also far higher than the tuning simulator predicted (up to 726 moves/min against at most 40).

Next steps are repeating the main comparisons on real machines with a better network and truly different hardware, adding health checks and draining, testing failures, repeating each configuration at least 5 times, and exploring why RIF hunts. Using the kernel's own queue as a signal would also make PSI usable.

In conclusion, a stateless balancer with second hops can move load between unequal servers without breaking connections, even over a real network, and a simple feedback controller finds tables close to one built from measured capacities. A controller is not better than a static table that knows what will happen, and over a slow network it may not recover in time. Its benefit is that it needs no such knowledge. Our most useful lesson was

about method: most of our surprises came from the measuring setup and from our own overclaims, not from the data path.

References

1. D. E. Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer," USENIX NSDI, 2016.
2. N. Shirokov and R. Dasineni, "Open-sourcing Katran, a scalable network load balancer," Facebook Engineering, 2018.
3. Cloudflare, "Unimog: Cloudflare's edge load balancer," Cloudflare Blog, 2020.
4. V. Olteanu et al., "Stateless Datacenter Load-balancing with Beamer," USENIX NSDI, 2018.
5. GitHub, "GLB: GitHub's open source load balancer," GitHub Engineering, 2018.
6. B. Wydrowski et al., "Load is not what you should balance: Introducing Prequal," USENIX NSDI, 2024.
7. T. Høiland-Jørgensen et al., "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel," ACM CoNEXT, 2018.
8. G. Heiser, "Systems Benchmarking Crimes," gernot-heiser.org.
9. J. Dean and L. A. Barroso, "The Tail at Scale," Communications of the ACM, 2013.